IEM UTM STUDENT SECTION

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

IEM Connect

Sprint 4 Presentation

Presented by: Group GGEZ

Date: 29 December 2025

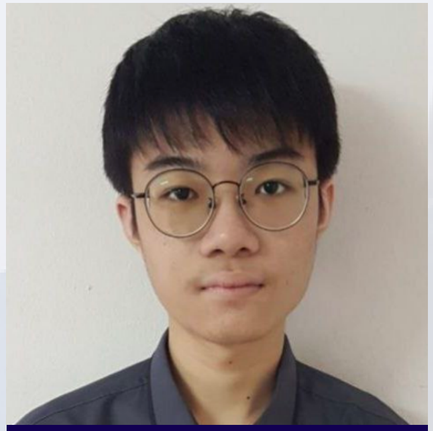
Play

Our Team

Chuah Chun Yi

Scrum Master

Lead Coordination & Stakeholder Communication



Chong Jun Hong

Developer 1

Backend Development

Muhammad Ariff bin Baharuddin

Developer 2

Frontend Development

Huzsa Syahyzal Haiqal bin Hussain

Developer 3

Frontend Development

Hrishi Vengalsetti

Developer 4

Backend Development

Sprint Overview

Sprint Goal

- To deliver AI Chatbot Interaction, Certificate Generation, and Logout & Session Security features.

Sprint Duration

- 2 Weeks (Analysis + Design + Development + Testing)

MVP Objective

- Deliver AI Chatbot Interaction deferred from previous sprint.
- Implement Certificate Generation for event participants.
- Improve system security through Logout & Session handling.

Sprint Backlog Summary

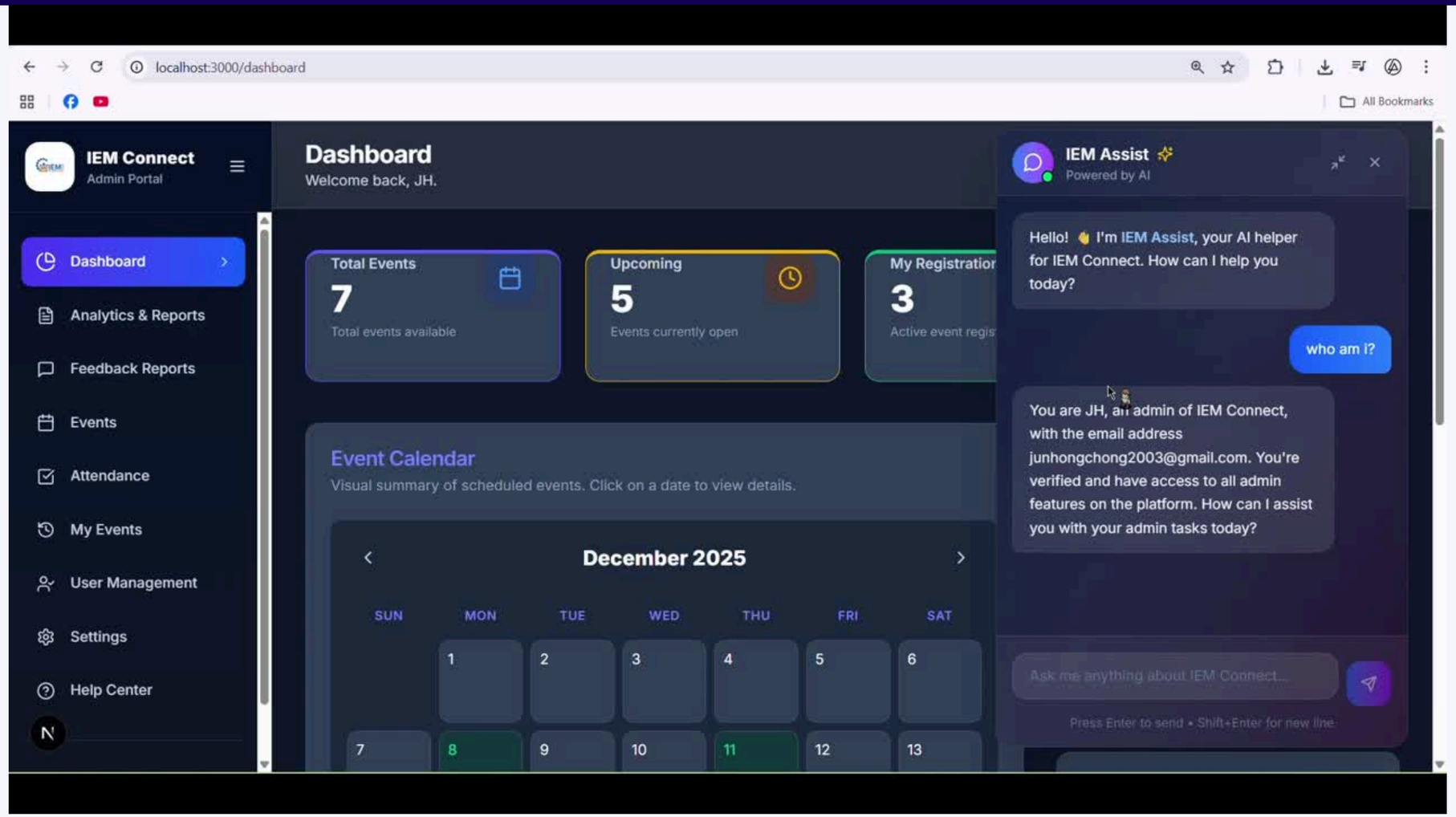
Main User Stories Implemented:

1. As a member, I want to chat with the AI chatbot so that I can inquire information about IEM and its events.
2. As an admin, I want to generate e-certificates automatically so that I can reduce manual certificate work.
3. As a member, I want to log out and clear session tokens so that my account remains secure.

Deliverables:

- AI Chatbot Interaction
- Certificate Generation
- Logout & Session Security

MVP Demonstration



Client Feedback

Key Comments Received:

1. Users should be able to view all obtained certificates in one centralized page.
2. Satisfied for AI chatbot can differentiate between admin and members.
3. Minor UI inconsistencies observed across upper sections of pages.
4. Overall feedback is very positive and the system effectively supports IEM UTM event management.

Action Items for Next Sprint:

- Complete remaining UI consistency refinements.
- Perform post-deployment testing and refinements.

Code Hightlights

Core Components

backend/controllers/chatbotController.js:

- Handle AI chatbot requests from members and admin
- Integrate with multiple AI providers to enhance reliability
- Enhance security rules to secure data privacy

backend/controllers/certificateController.js:

- Handle PDF certificates generation for event participants
- Generate certificates correspond with participant and event details

backend/controllers/authController.js:

- Handle user authentication and authorization
- Manage secure logout and session validation
- Terminate active user session on client side once logout

Example Logic

1 AI Chatbot Interaction Logic

```typescript
export const handleChatbotMessage = async (req, res) => {
  console.log("\n========== CHATBOT REQUEST ==========");

  try {
    const { message, conversationHistory = [] } = req.body;
    const userId = req.user.id;

    if (!message || typeof message !== "string" || message.trim().length === 0) {
      return res.status(400).json({ error: "Message is required" });
    }

    console.log(`📨 Message: "${message}"`);
    console.log(`👤 User ID: ${userId}`);

    // Get context from database
    const context = await getChatbotContext(userId, message);
    const contextString = formatContextForAI(context);
    const userRole = context.userInfo?.role || 'member';

    console.log(`👤 User: ${context.userInfo?.name} (${userRole})`);

    // Build system prompt
    const systemPrompt = buildPromptWithContext(message, conversationHistory, contextString, context.userInfo);

    console.log(`📊 Total prompt size: ${systemPrompt.length + message.length} chars`);

    let aiResponse;

    // Try Groq first, fallback to Gemini, then to rule-based
    try {
      aiResponse = await callGroq(systemPrompt, message);
    } catch (groqError) {
      console.log("⚠️ Groq failed, trying Gemini...");

      try {
        aiResponse = await callGemini(systemPrompt, message);
      } catch (geminiError) {
        console.log("⚠️ Gemini also failed, using fallback...");
        aiResponse = generateFallbackResponse(message, userRole);
      }
    }

    console.log("========== RESPONSE SENT ==========\n");

    res.json({
      message: aiResponse.trim(),
      timestamp: new Date().toISOString(),
    });
  } catch (error) {
    console.error("❌ Chatbot error:", error);
    res.status(500).json({
      error: "Failed to process message",
      details: error.message,
    });
  }
};
```

```typescript
const callGroq = async (systemPrompt, userMessage) => {
  console.log("🚀 Calling Groq (llama-3.3-70b-versatile)...");
  console.log(`   System prompt length: ${systemPrompt.length} chars`);
  console.log(`   User message: "${userMessage.substring(0, 50)}..."`);

  try {
    const client = getGroqClient();

    const completion = await client.chat.completions.create({
      model: "llama-3.3-70b-versatile",
      messages: [
        { role: "system", content: systemPrompt },
        { role: "user", content: userMessage },
      ],
      temperature: 0.7,
      max_tokens: 512,
      top_p: 1,
    });

    const response = completion.choices[0]?.message?.content;
    if (!response) {
      throw new Error("Empty response from Groq");
    }

    console.log("✅ Groq response received:", response.substring(0, 50) + "...");
    return response;
  } catch (error) {
    console.error("❌ Groq error details:", {
      message: error.message,
      status: error.status,
      code: error.code,
    });
    throw error;
  }
};

/**
 * Call Gemini API (Fallback Provider)
 */
const callGemini = async (systemPrompt, userMessage) => {
  console.log("🔄 Calling Gemini (gemini-1.5-flash)...");

  try {
    const client = getGeminiClient();
    const model = client.getGenerativeModel({ model: "gemini-1.5-flash" });

    const fullPrompt = `${systemPrompt}\n\nUser question: ${userMessage}\n\nAssistant response:`;

    const result = await model.generateContent(fullPrompt);
    const response = result.response.text();

    if (!response) {
      throw new Error("Empty response from Gemini");
    }

    console.log("✅ Gemini response received:", response.substring(0, 50) + "...");
    return response;
  } catch (error) {
    console.error("❌ Gemini error details:", {
      message: error.message,
      status: error.status,
      errorDetails: error.errorDetails,
    });
    throw error;
  }
};
```

- Receive user message and generate AI response based on acknowledged users' role
- Call to different AI providers in case one failed

Example Logic

2 Certificate Generation Logic

```typescript
export const generateCertificate = async (req, res) => {
  try {
    const { eventId } = req.params;
    const userId = req.user.id;

    // 1. Verify user attended the event
    const registration = await EventRegistration.findOne({
      where: { user_id: userId, event_id: eventId },
      include: [
        { model: User, as: "user" },
        { model: Event, as: "event" },
      ],
    });

    if (!registration) {
      return res.status(404).json({ error: "Registration not found" });
    }

    // 2. Check if user actually attended (has attendance record)
    const attendance = await Attendance.findOne({
      where: { registration_id: registration.id },
    });

    if (!attendance) {
      return res.status(403).json({
        error:
          "Certificate only available for participants who attended the event",
      });
    }

    // 3. Load PDF template
    const templateBytes = await loadTemplate();
    const pdfDoc = await PDFDocument.load(templateBytes);

    // 4. Get first page and dimensions
    const pages = pdfDoc.getPages();
    const firstPage = pages[0];
    const { width, height } = firstPage.getSize();

    // 5. Embed fonts
    const helveticaFont = await pdfDoc.embedFont(StandardFonts.Helvetica);
    const helveticaBoldFont = await pdfDoc.embedFont(StandardFonts.HelveticaBold);

    // 6. Helper function to convert to Title Case
    const toTitleCase = (str) => {
      return str.replace(/\w\S*/g, (txt) => {
        return txt.charAt(0).toUpperCase() + txt.substr(1).toLowerCase();
      });
    };

    // Helper function to get ordinal suffix (1st, 2nd, 3rd, etc.)
    const getOrdinalSuffix = (day) => {
      if (day > 3 && day < 21) return 'th';
      switch (day % 10) {
        case 1: return 'st';
        case 2: return 'nd';
        case 3: return 'rd';
        default: return 'th';
      }
    };
```

```typescript
    // 7. Format dates
    const eventDateObj = new Date(registration.event.start_date);
    const day = eventDateObj.getDate();
    const month = eventDateObj.toLocaleDateString("en-US", { month: "long" });
    const year = eventDateObj.getFullYear();
    const eventDate = `${day}${getOrdinalSuffix(day)} ${month} ${year}`;

    // 8. Add text fields with precise A4 Landscape coordinates
    // A4 Landscape: Width = 842, Height = 595 (Y=0 is bottom)

    // Participant Name (Title Case, centered, larger, bold)
    const participantName = toTitleCase(registration.user.name);
    const nameFontSize = 30;
    const nameTextWidth = helveticaBoldFont.widthOfTextAtSize(
      participantName,
      nameFontSize
    );
    firstPage.drawText(participantName, {
      x: width / 2 - nameTextWidth / 2, // Center horizontally
      y: 260, // Fixed Y coordinate for A4 landscape
      size: nameFontSize,
      font: helveticaBoldFont,
      color: rgb(0, 0, 0),
    });

    // Event Title Line ("For participating in...")
    const eventTitle = registration.event.title;
    const eventTitleText = `For participating in ${eventTitle}`;
    const eventTitleFontSize = 16;
    const eventTitleTextWidth = helveticaFont.widthOfTextAtSize(
      eventTitleText,
      eventTitleFontSize
    );
    firstPage.drawText(eventTitleText, {
      x: width / 2 - eventTitleTextWidth / 2,
      y: 230, // Fixed Y coordinate
      size: eventTitleFontSize,
      font: helveticaFont,
      color: rgb(0, 0, 0),
    });

    // Date Line
    const dateText = `on ${eventDate}`;
    const dateFontSize = 16;
    const dateTextWidth = helveticaFont.widthOfTextAtSize(
      dateText,
      dateFontSize
    );
    firstPage.drawText(dateText, {
      x: width / 2 - dateTextWidth / 2,
      y: 200, // Fixed Y coordinate
      size: dateFontSize,
      font: helveticaFont,
      color: rgb(0, 0, 0),
    });
```

```typescript
    // Event Title Line ("For participating in...")
    const eventTitle = registration.event.title;
    const eventTitleText = `For participating in ${eventTitle}`;
    const eventTitleFontSize = 16;
    const eventTitleTextWidth = helveticaFont.widthOfTextAtSize(
      eventTitleText,
      eventTitleFontSize
    );
    firstPage.drawText(eventTitleText, {
      x: width / 2 - eventTitleTextWidth / 2,
      y: 230, // Fixed Y coordinate
      size: eventTitleFontSize,
      font: helveticaFont,
      color: rgb(0, 0, 0),
    });

    // Date Line
    const dateText = `on ${eventDate}`;
    const dateFontSize = 16;
    const dateTextWidth = helveticaFont.widthOfTextAtSize(
      dateText,
      dateFontSize
    );
    firstPage.drawText(dateText, {
      x: width / 2 - dateTextWidth / 2,
      y: 200, // Fixed Y coordinate
      size: dateFontSize,
      font: helveticaFont,
      color: rgb(0, 0, 0),
    });

    // 8. Generate PDF bytes
    const pdfBytes = await pdfDoc.save();

    // 9. Stream to client
    const safeEventTitle = registration.event.title.replace(/[^a-zA-Z0-9]/g, "_");
    const safeUserName = registration.user.name.replace(/[^a-zA-Z0-9]/g, "_");
    const filename = `Certificate_${safeEventTitle}_${safeUserName}.pdf`;

    res.setHeader("Content-Type", "application/pdf");
    res.setHeader(
      "Content-Disposition",
      `inline; filename="${filename}"`
    );
    res.setHeader("Content-Length", pdfBytes.length);

    res.send(Buffer.from(pdfBytes));
  } catch (error) {
    console.error("Certificate generation error:", error);

    if (error.message.includes("template not found")) {
      return res.status(500).json({
        error: "Certificate template not found. Please contact administrator."
      });
    }

    res.status(500).json({ error: "Failed to generate certificate" });
  }
};
```

- Verify event registration and event attendance of user
- Populate certificate details on template and generate PDF certificate

Example Logic

3 Logout & Session Security Logic

```javascript
export const logout = async (req, res) => {
  try {
    // In a real implementation, you would invalidate the token here
    // For JWT, this typically involves adding the token to a blacklist
    res.status(200).json({ message: "Logged out successfully" });
  } catch (error) {
    console.error("Logout error:", error);
    res.status(500).json({ error: "Logout failed" });
  }
};
```

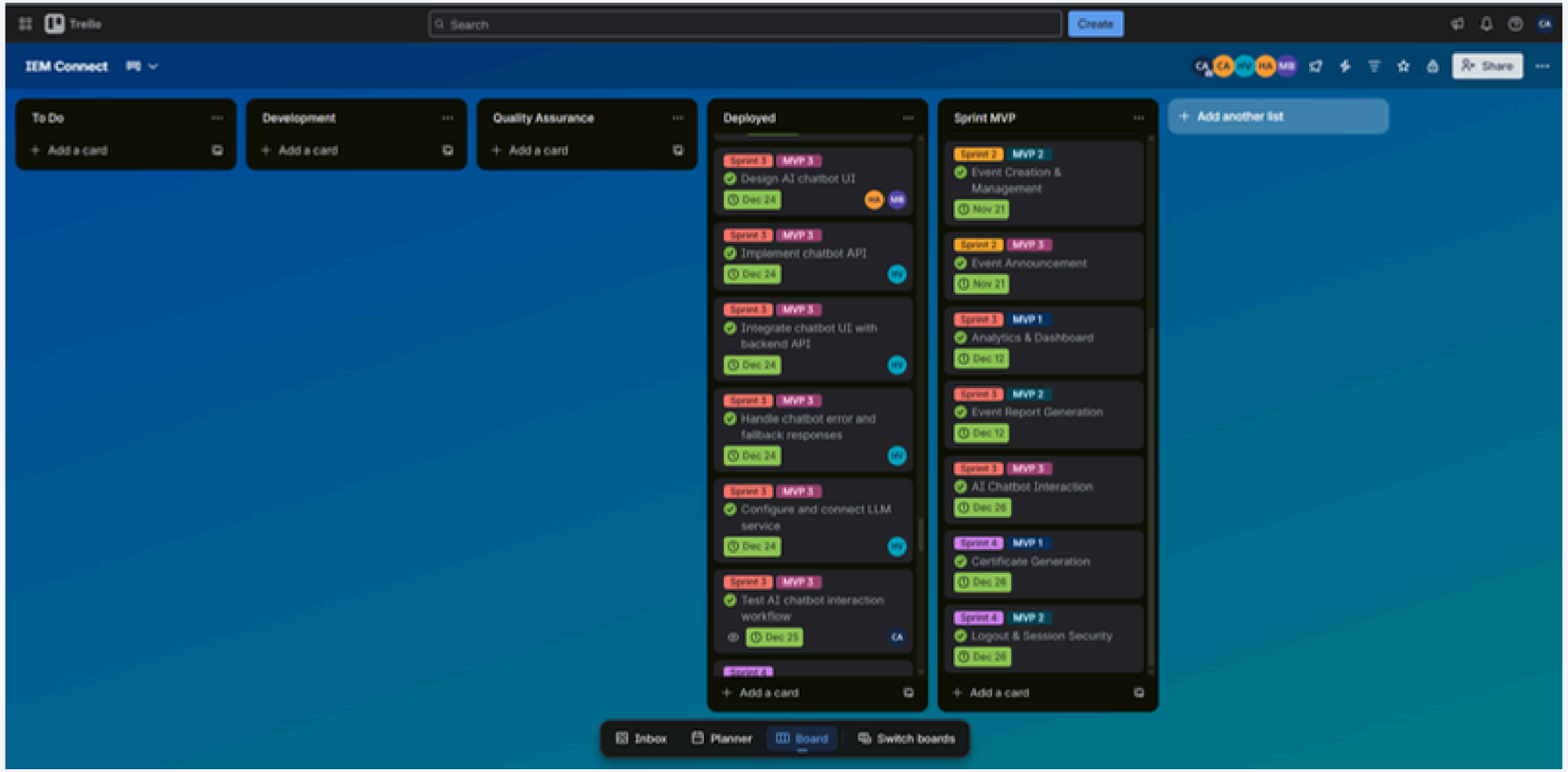
```javascript
const generateTokens = (user) => {
  const accessToken = jwt.sign(
    { id: user.id, role: user.role, name: user.name },
    process.env.JWT_SECRET,
    { expiresIn: "1h" }
  );

  const refreshToken = jwt.sign(
    { id: user.id },
    process.env.JWT_REFRESH_SECRET,
    { expiresIn: "7d" }
  );

  return { accessToken, refreshToken };
};
```

```javascript
export const verifySession = async (req, res) => {
  try {
    const user = await User.findByPk(req.user.id, {
      attributes: [
        "id",
        "name",
        "email",
        "role",
        "membership_number",
        "matric_number",
        "faculty",
        "bio",
        "avatar_url",
      ],
    });

    if (!user) {
      return res.status(404).json({ error: "User not found" });
    }

    res.status(200).json({
      user: {
        id: user.id,
        name: user.name,
        email: user.email,
        role: user.role,
        membership_number: user.membership_number,
        matric_number: user.matric_number,
        faculty: user.faculty,
        bio: user.bio,
        avatar_url: user.avatar_url,
      },
    });
  } catch (error) {
    console.error("Session verification error:", error);
    res.status(500).json({ error: "Session verification failed" });
  }
};
```

- Ensure safe session validation through secured tokens generated
- Handle secure logout and session termination

Project Performance Metrics

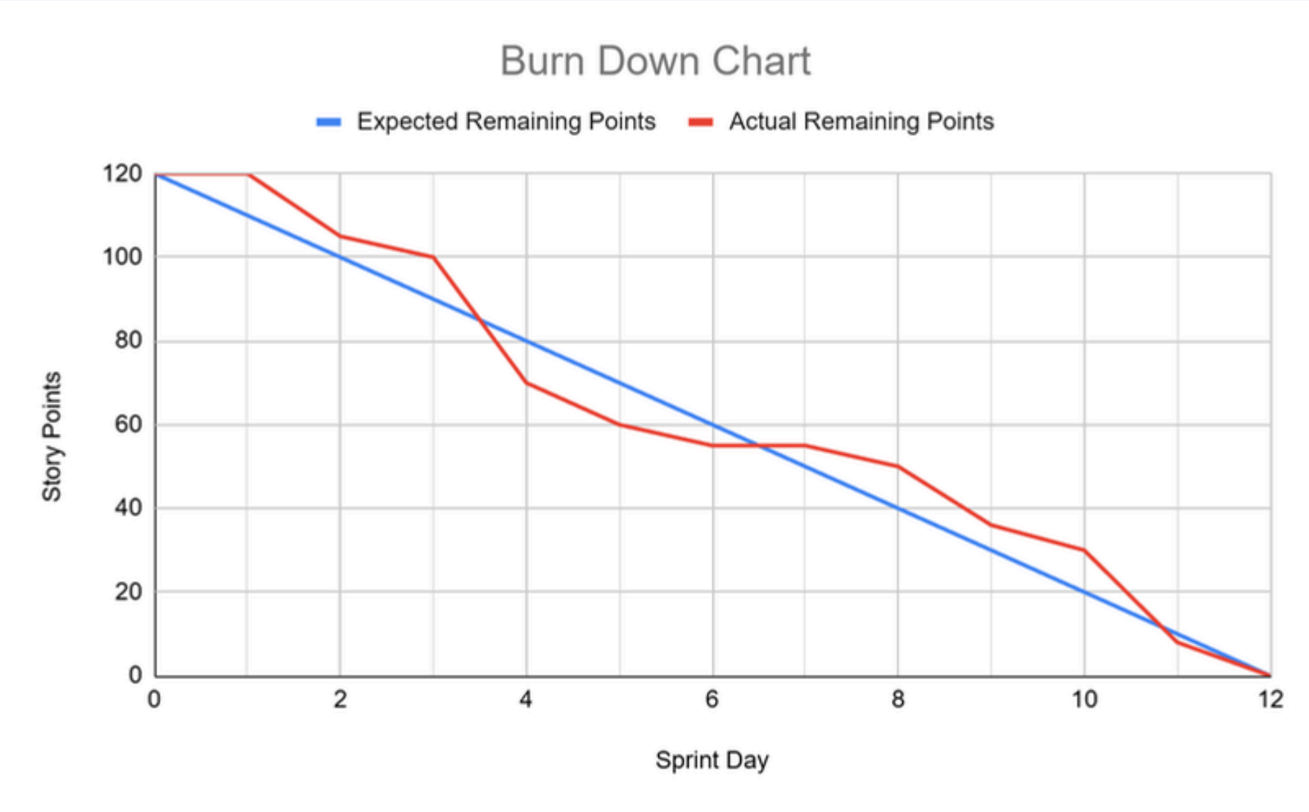
Kanban Board



Sprint 4
Total tasks: 28
Completed: 28
In progress: 0

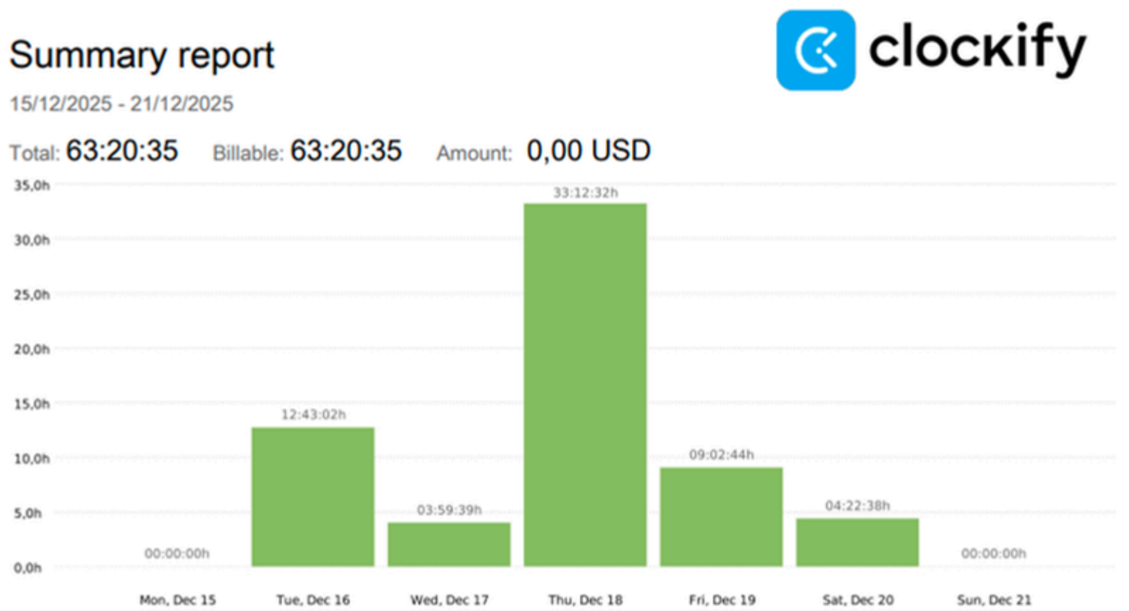
Project
Total tasks: 11
Completed: 11
In progress: 0

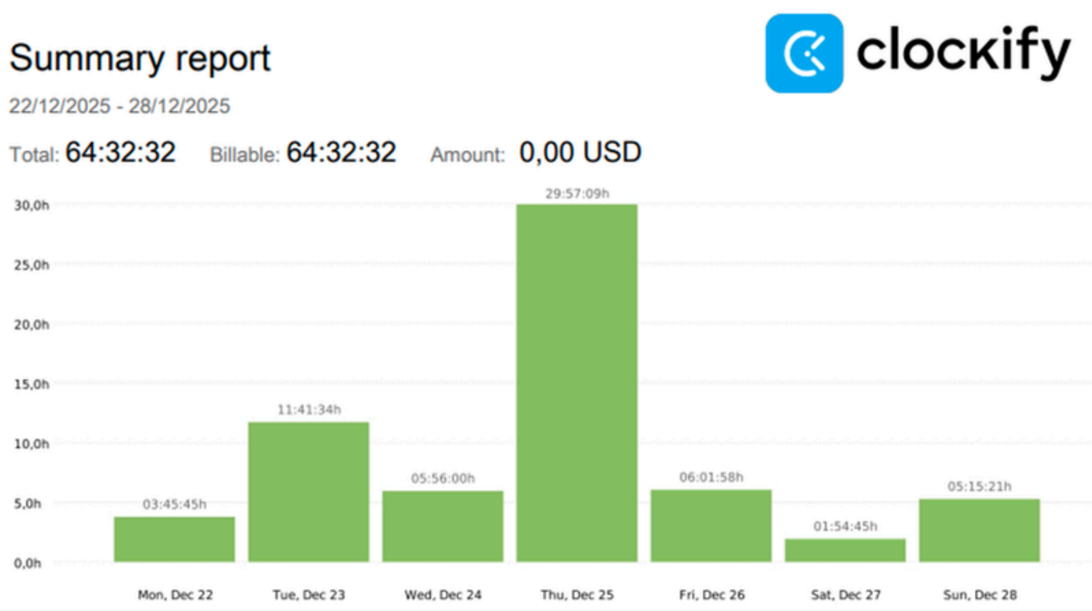
Burndown Chart



- Sprint completed with all planned tasks delivered
- Actual progress aligns closely with expected trend

Clockify Summary





Total hours logged: 127.88 hours
Average per member: 25.58 hours

Next Sprint Focus

Fine-Tuning Product Focus Area:

- Fix minor UI and website hosting bugs
- Improve system stability and performance
- Refine overall user experience and consistency

Target Outcome:

- Stable and bug-minimized system
- Improved usability and UI consistency
- System ready for final acceptance and deployment

Conclusion

Summary:

- Sprint 4 successfully delivered AI Chatbot Interaction, Certificate Generation and Logout & Session Security features.
- The system is now functionally complete, with core MVP fully achieved.

Next Steps:

- Fix remaining minor UI and hosting-related bugs.
- Perform final system testing and stability checks.
- Prepare the system for final acceptance and deployment.

Thank You

Contact Us

Chuah Chun Yi

Scrum Master of Group GGEZ

+60 1157678402

iemconnect1@gmail.com

Sprint Presentation Slide Link

Link:

- Sprint 1
- Sprint 2
- Sprint 3
- Sprint 4